

รายงาน Lab 10: Spring WebFlux & WebClient

วิชา: CP353002 หลักการออกแบบและพัฒนาซอฟต์แวร์

ชื่อ-นามสกุล: นายอนันต์เอกก์ ใหญ่พงศกร

รหัสนักศึกษา: 673380430-9

Section: 3

1. บทสรุปเชิงทฤษฎี (Theoretical Background)

1.1 Reactive Programming vs Blocking I/O

- **Blocking I/O (Thread-per-request):** โมเดลแบบเดิมที่ 1 Request จะจอง 1 Thread เพื่อประมวลผล หากต้องรอ I/O (เช่น Database หรือ External API) Thread นั้นจะถูกบล็อก (Wait) ส่งผลให้สิ้นเปลืองทรัพยากรเมื่อมี Traffic สูง
- **Reactive Programming (Non-blocking I/O):** โมเดลแบบ Asynchronous และ Event-driven ที่ใช้ทรัพยากร Thread จำนวนน้อย (Few Threads) ทำงานผ่าน Data Stream และ Event Loop เมื่อมี Request เข้ามา Thread จะไม่นั่งรอผลลัพธ์ แต่จะไปทำงานอื่นต่อ แล้วกลับมาจัดการเมื่อได้รับ Data Signal ทำให้รองรับ Concurrency สูงได้ดียิ่งขึ้น

1.2 Mono vs Flux

- **Mono:** เป็น Publisher ใน Project Reactor ที่ส่งออกข้อมูลจำนวน 0 ถึง 1 ค่า เหมาะสำหรับงานที่คืนค่าผลลัพธ์เดียว เช่น findById(), save(), delete() หรือการคืนค่าสถานะ Mono<Void>
- **Flux:** เป็น Publisher ที่ส่งออกข้อมูลจำนวน 0 ถึง N ค่า (Data Stream) เหมาะสำหรับงานที่คืนค่าเป็นคอลเลกชันหรือต่อเนื่อง เช่น findAll(), การค้นหาตามหมวดหมู่ หรือ Event Streaming

1.3 การทำงานของ WebClient Method Chain

WebClient เป็น HTTP Client แบบ Non-blocking สำหรับเรียกใช้ REST API โดยมีลำดับ Method Chain ดังนี้:

1. **.get() / .post() / .delete()**: กำหนด HTTP Method ที่ต้องการส่ง Request
2. **.uri(...)**: กำหนด URL Path และ Path Variables สำหรับการเชื่อมต่อ
3. **.bodyValue(...)** (ถ้ามี): แนบ Body ข้อมูลสำหรับส่งไปยังเซิร์ฟเวอร์ (ใช้กับ POST/PUT)
4. **.retrieve()**: ส่งเริ่มส่ง Request และรอรับ Response Stream กลับมา
5. **.bodyToMono(Class) / .bodyToFlux(Class)**: แปลงข้อมูลใน Response Body ให้อยู่ในรูป Reactive Publisher (Mono หรือ Flux)

2. ซอร์สโค้ดการพัฒนาระบบ (Source Code Implementation)

2.1 ProductRepository.java

Java

```
// จัดเก็บข้อมูลแบบ In-Memory บน ConcurrentHashMap และจัดการสินค้าแบบ Mono/Flux
public class ProductRepository {
    private final Map<String, Product> store = new ConcurrentHashMap<>();

    public ProductRepository() {
        store.put("1", new Product("1", "iPhone 15 Pro (67XXXXXXX-X SEC 1)", "Electronics", "Apple", 50, 39900.0, "MEMBER"));
        store.put("2", new Product("2", "MacBook Air M3", "Electronics", "Apple", 20, 49900.0, "NONE"));
        store.put("3", new Product("3", "Samsung Galaxy S24", "Electronics", "Samsung", 30, 29900.0, "SEASONAL"));
    }

    public Mono<Product> findById(String id) {
        Product product = store.get(id);
        return product != null ? Mono.just(product) : Mono.empty();
    }

    public Flux<Product> findAll() {
        return Flux.fromIterable(store.values());
    }

    public Mono<Product> save(Product product) {
        store.put(product.getId(), product);
        return Mono.just(product);
    }

    public Mono<Void> deleteById(String id) {
        store.remove(id);
        return Mono.empty();
    }

    public Flux<Product> findByCategory(String category) {
        return findAll().filter(p ->
            p.getCategory().equalsIgnoreCase(category));
    }
}
```

2.2 ProductService.java

Java

```
// จัดการ Business Logic และความคุม Error Handling / UUID Generation
@Service
public class ProductService {
    private final ProductRepository repository;

    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }

    public Mono<Product> getById(String id) {
        return repository.findById(id)
            .switchIfEmpty(Mono.error(new RuntimeException("Product not
found: " + id)));
    }

    public Flux<Product> getAll() {
        return repository.findAll();
    }

    public Mono<Product> save(Product product) {
        if (product.getId() == null) {
            product.setId(UUID.randomUUID().toString());
        }
        return repository.save(product);
    }

    public Mono<Void> delete(String id) {
        return repository.deleteById(id);
    }

    public Flux<Product> getByCategory(String category) {
        return repository.findByCategory(category);
    }

    public Mono<Double> getDiscountedPrice(String id) {
        return getById(id).map(Product::getDiscountedPrice);
    }
}
```

2.3 ProductController.java

Java

```
// REST Controller สำหรับรับ Request และส่งคืน Mono/Flux Endpoints
@RestController
@RequestMapping("/products")
public class ProductController {

    private final ProductService service;

    public ProductController(ProductService service) {
        this.service = service;
    }

    @GetMapping("/{id}")
    public Mono<Product> getById(@PathVariable String id) {
        return service.getById(id);
    }

    @GetMapping
    public Flux<Product> getAll() {
        return service.getAll();
    }

    @PostMapping
    public Mono<Product> save(@RequestBody Product product) {
        return service.save(product);
    }

    @DeleteMapping("/{id}")
    public Mono<Void> delete(@PathVariable String id) {
        return service.delete(id);
    }

    @GetMapping("/category/{category}")
    public Flux<Product> getByCategory(@PathVariable String category) {
        return service.getByCategory(category);
    }

    @GetMapping("/{id}/price")
    public Mono<Double> getDiscountedPrice(@PathVariable String id) {
        return service.getDiscountedPrice(id);
    }
}
```

3. ผลการทดสอบการทำงาน (Test Results & Screenshots)

3.1 ผลการทดสอบ Unit Test (mvn test)

- **สถานะ: BUILD SUCCESS** (Tests run: 6, Failures: 0, Errors: 0, Skipped: 0)

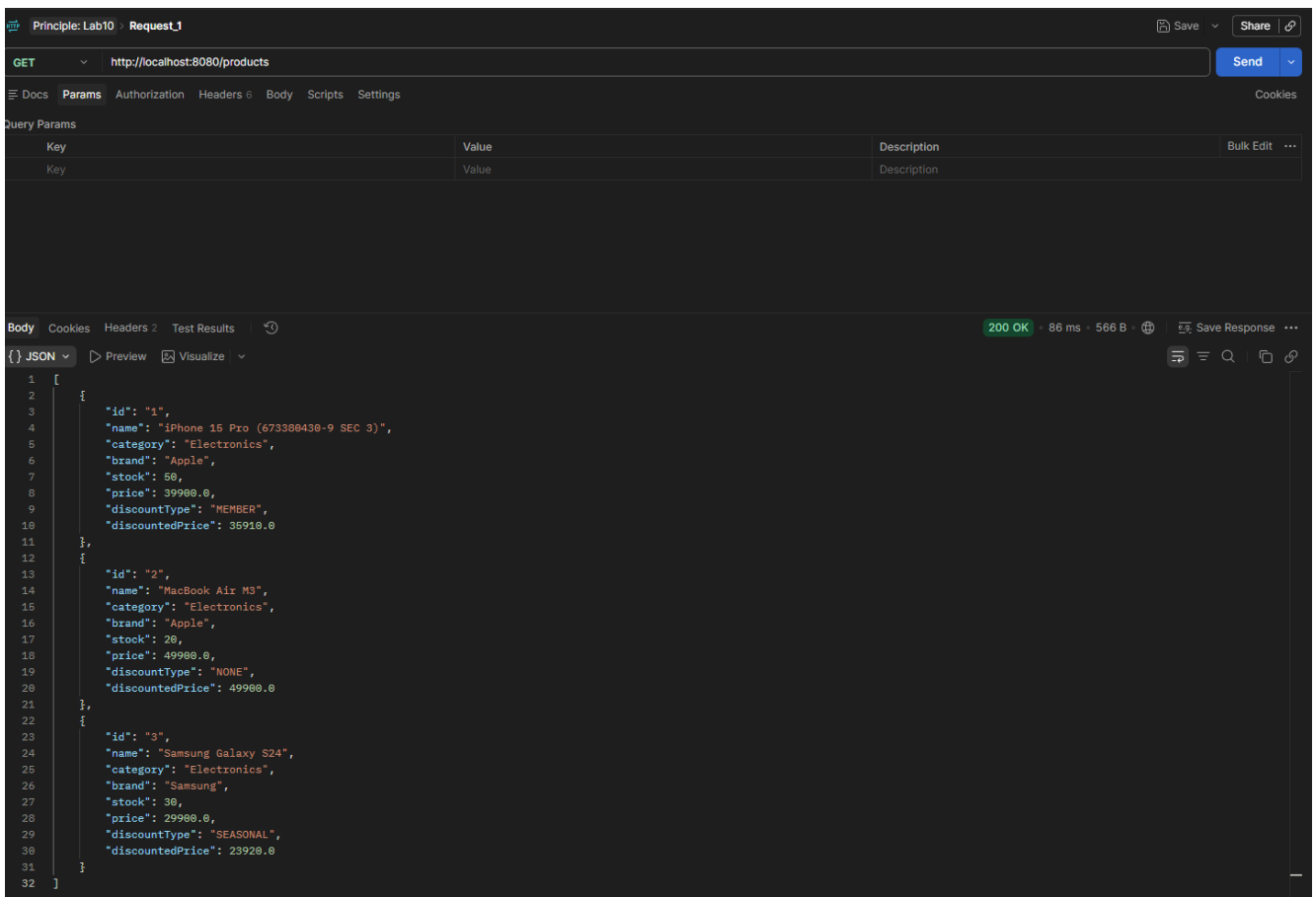
```

\O-Spring>
=====
:: Spring Boot ::                (v3.3.0)

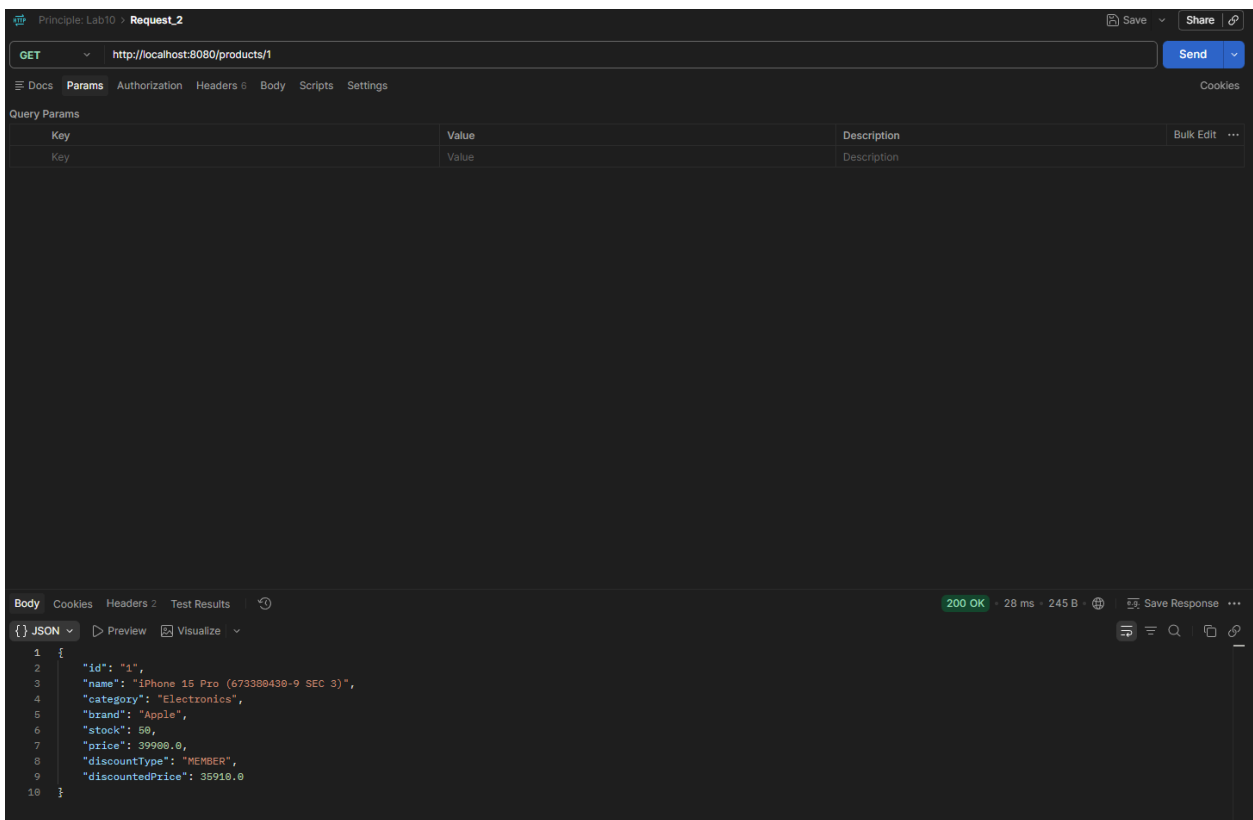
2026-09-20T09:43:07:524+07:00 INFO 9512 --- [lab10-webflux] [
Design\Labs\Lab10\Lab10-673380430-9-sec3)
2026-09-20T09:43:07:525+07:00 DEBUG 9512 --- [lab10-webflux] [
2026-09-20T09:43:07:526+07:00 INFO 9512 --- [lab10-webflux] [
2026-09-20T09:43:09:471+07:00 INFO 9512 --- [lab10-webflux] [
WARNING: A Java agent has been loaded dynamically (C:\Users\anane\.m2\repository\net\bytebuddy\byte-buddy-agent\1.14.16\byte-buddy-agent-1.14.16.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.136 s -- in com.example.lab10.Lab10ApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 7.790 s
[INFO] Finished at: 2026-09-20T09:43:10+07:00
[INFO] -----
```

3.2 ผลการทดสอบ Endpoints (ผ่าน Browser / Postman)

1. GET /products: ดึงข้อมูลสินค้าทั้งหมด แสดงผลชื่อสินค้าที่มีชื่อและรหัสนักศึกษา



2. GET /products/1: ค้นหาสินค้าตาม ID



Principle: Lab10 > Request_2

GET http://localhost:8080/products/1

Send

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

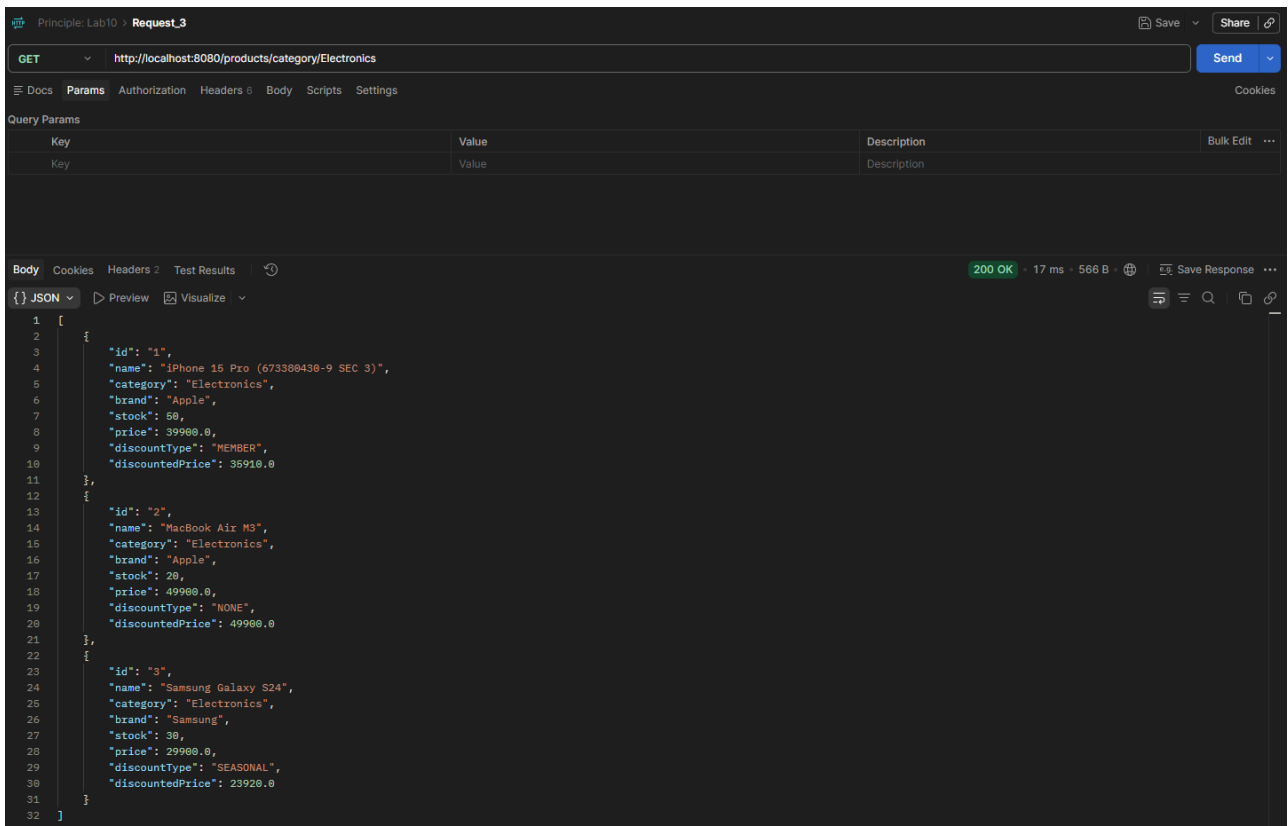
Body Cookies Headers 2 Test Results

200 OK · 28 ms · 245 B · Save Response

JSON Preview Visualize

```
1 {
2   "id": "1",
3   "name": "iPhone 15 Pro (673388438-9 SEC 3)",
4   "category": "Electronics",
5   "brand": "Apple",
6   "stock": 58,
7   "price": 39900.0,
8   "discountType": "MEMBER",
9   "discountedPrice": 35910.0
10 }
```

3. GET /products/category/Electronics: กรองสินค้าเฉพาะหมวดหมู่ Electronics



Principle: Lab10 > Request_3

GET http://localhost:8080/products/category/Electronics

Send

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies Headers 2 Test Results

200 OK · 17 ms · 566 B · Save Response

JSON Preview Visualize

```
1 [
2   {
3     "id": "1",
4     "name": "iPhone 15 Pro (673388438-9 SEC 3)",
5     "category": "Electronics",
6     "brand": "Apple",
7     "stock": 58,
8     "price": 39900.0,
9     "discountType": "MEMBER",
10    "discountedPrice": 35910.0
11  },
12  {
13    "id": "2",
14    "name": "MacBook Air M3",
15    "category": "Electronics",
16    "brand": "Apple",
17    "stock": 20,
18    "price": 49900.0,
19    "discountType": "NONE",
20    "discountedPrice": 49900.0
21  },
22  {
23    "id": "3",
24    "name": "Samsung Galaxy S24",
25    "category": "Electronics",
26    "brand": "Samsung",
27    "stock": 30,
28    "price": 29900.0,
29    "discountType": "SEASONAL",
30    "discountedPrice": 23920.0
31  }
32 ]
```

4. GET /products/1/price: แสดงราคาหลังหักส่วนลด

The screenshot shows a REST client interface for a request named "Request_4". The method is GET and the URL is http://localhost:8080/products/1/price. The response status is 200 OK, with a response time of 13 ms and a body size of 79 B. The response body is displayed in JSON format as follows:

```
1 35910.0
```

5. POST /products: ทดสอบการบันทึกข้อมูลสินค้าใหม่

The screenshot shows a REST client interface for a request named "Request_5". The method is POST and the URL is http://localhost:8080/products. The request body is in raw format and contains the following JSON:

```
1 {
2   "name": "iPad Pro",
3   "category": "Electronics",
4   "brand": "Apple",
5   "stock": 10,
6   "price": 32900.0,
7   "discountType": "NONE"
8 }
```

The response status is 200 OK, with a response time of 79 ms and a body size of 253 B. The response body is displayed in JSON format as follows:

```
1 {
2   "id": "5c4c068a-d78e-4b30-b553-a64545f6a71b",
3   "name": "iPad Pro",
4   "category": "Electronics",
5   "brand": "Apple",
6   "stock": 10,
7   "price": 32900.0,
8   "discountType": "NONE",
9   "discountedPrice": 32900.0
10 }
```


6. DELETE /products/1: ทดสอบการลบข้อมูลสินค้า

Principle: Lab10 > Request_6

DELETE http://localhost:8080/products/1

Docs Params Authorization Headers 8 Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Schema Beautify

```
1 {
2   "name": "iPad Pro",
3   "category": "Electronics",
4   "brand": "Apple",
5   "stock": 10,
6   "price": 32900.0,
7   "discountType": "NONE"
8 }
```

Body Cookies Headers 1 Test Results

200 OK · 11 ms · 40 B · Save Response

Raw Preview Visualize

1